

BookTrack

Sistema de Biblioteca Pessoal

Documentação Técnica — Avaliação Prática
Disciplina de Algoritmos e Programação

1. Visão Geral do Sistema

O BookTrack é um sistema de linha de comando (terminal) desenvolvido em Python 3, que permite a leitores organizarem sua biblioteca pessoal. O sistema gerencia dois tipos de entidades principais: **usuários** e **livros**, permitindo cadastro, edição, remoção/exclusão, login e exibição de dados, além de um relatório de progresso de leitura construído a partir de uma matriz.

Os dados são persistidos em arquivos CSV (`dados/usuarios.csv` e `dados/livros.csv`), lidos e escritos exclusivamente pelo módulo `arquivos.py`.

1.1 Organização do Código por Responsabilidade

Módulo	Responsabilidade
<code>main.py</code>	Ponto de entrada; controla o menu e a navegação entre telas
<code>usuarios.py</code>	Regras de negócio de usuários: cadastro, edição, exclusão, login, listagem
<code>livros.py</code>	Regras de negócio de livros: cadastro, edição, remoção, listagem
<code>relatorios.py</code>	Geração do relatório de progresso de leitura usando uma matriz
<code>arquivos.py</code>	Única camada de acesso a disco (leitura/escrita dos arquivos CSV)
<code>validacoes.py</code>	Validação de entradas digitadas no terminal (tipos e formatos)

2. Estruturas de Dados

2.1 Vetores (Listas)

O vetor `usuarios` é uma lista de dicionários, em que cada posição representa um usuário cadastrado. O mesmo padrão é usado para o vetor `livros`. A função `obter_vetor_titulos()` demonstra um vetor unidimensional simples, extraíndo apenas os títulos dos livros.

Estrutura do dicionário de usuário:

Campo	Tipo
nome	string
username	string
senha	string
nascimento	int (formato DDMMAAAA, ex: 20071980)

Estrutura do dicionário de livro:

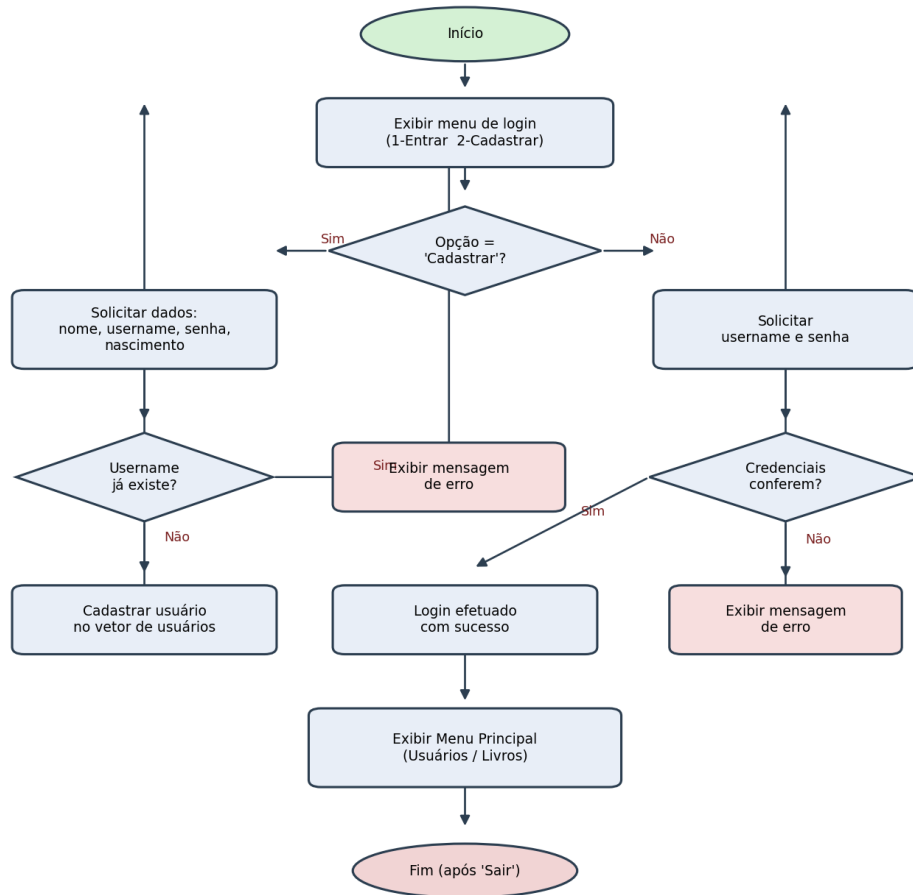
Campo	Tipo
titulo	string
autor	string
editora	string
edicao	int
ano	int
isbn	int
status	string
paginas_totais	int
paginas_lidas	int

2.2 Matriz

O módulo `relatorios.py` constrói explicitamente uma **matriz** (lista de listas) através da função `gerar_matriz_relatorio()`. Cada linha da matriz representa um livro, e as colunas seguem a ordem: `[titulo, status, paginas_lidas, paginas_totais, percentual]`. A função `exibir_matriz_relatorio()` percorre a matriz (linha por linha e coluna por coluna) para exibi-la formatada no console.

3. Fluxograma — Fluxo de Login

O fluxograma abaixo descreve o fluxo principal de login, incluindo o caminho alternativo de cadastro de um novo usuário e o tratamento de credenciais inválidas.



4. Documentação das Funções e Procedimentos

4.1 Módulo arquivos.py

garantir_pasta_dados()

Responsabilidade: Garantir que a pasta de dados exista.

Entradas: nenhuma

Saídas: nenhuma

ler_csv(caminho, campos)

Responsabilidade: Ler um CSV e devolver um vetor de dicionários.

Entradas: caminho (string), campos (list)

Saídas: list — vetor de dicionários

escrever_csv(caminho, campos, registros)

Responsabilidade: Sobrescrever um CSV com o vetor em memória.

Entradas: caminho (string), campos (list), registros (list)

Saídas: nenhuma

carregar_usuarios() / carregar_livros()

Responsabilidade: Carregar o vetor correspondente a partir do disco.

Entradas: nenhuma

Saídas: list — vetor de dicionários

salvar_usuarios(usuarios) / salvar_livros(livros)

Responsabilidade: Persistir o vetor correspondente em disco.

Entradas: usuarios ou livros (list)

Saídas: nenhuma

4.2 Módulo usuarios.py

registrar_usuario(usuarios, nome, username, senha, nascimento)

Responsabilidade: Cadastrar um novo usuário, impedindo username duplicado.

Entradas: usuarios (list), nome/username/senha/nascimento (string)

Saídas: bool — True se cadastrado, False se username já existe

buscar_usuario_por_username(usuarios, username)

Responsabilidade: Localizar um usuário pelo username, sem diferenciar maiúsculas de minúsculas.

Entradas: usuarios (list), username (string)

Saídas: dict ou None

editar_usuario(usuarios, username, novo_nome, novo_username, nova_senha, novo_nascimento)

Responsabilidade: Atualizar dados de um usuário existente, incluindo o username (verificando que o novo username não pertença a outro usuário).

Entradas: usuarios (list), username (string), novo_nome/novo_username/nova_senha (string, opcionais), novo_nascimento (int, opcional)

Saídas: string — 'ok', 'nao_encontrado' ou 'username_em_uso'

excluir_usuario(usuarios, username)

Responsabilidade: Remover um usuário do vetor.

Entradas: usuarios (list), username (string)

Saídas: bool — True se removido, False se não encontrado

autenticar_login(usuarios, username, senha)

Responsabilidade: Validar credenciais de login.

Entradas: usuarios (list), username (string), senha (string)

Saídas: bool — True se credenciais conferem

listar_usuarios(usuarios)

Responsabilidade: Exibir todos os usuários cadastrados no console.

Entradas: usuarios (list)

Saídas: nenhuma (impressão no console)

4.3 Módulo livros.py

cadastrar_livro(livros, titulo, autor, editora, edicao, ano, isbn, status, paginas_totais, paginas_lidas)

Responsabilidade: Cadastrar um novo livro, impedindo ISBN duplicado.

Entradas: livros (list), demais campos do livro (string/int)

Saídas: bool — True se cadastrado, False se ISBN já existe

buscar_livro_por_isbn(livros, isbn)

Responsabilidade: Localizar um livro pelo ISBN.

Entradas: livros (list), isbn (string)

Saídas: dict ou None

editar_livro(livros, isbn, novo_status, novas_paginas_lidas)

Responsabilidade: Atualizar status e/ou progresso de leitura de um livro.

Entradas: livros (list), isbn (string), novo_status (string, opcional), novas_paginas_lidas (int, opcional)

Saídas: bool — True se atualizado, False se não encontrado

remover_livro(livros, isbn)

Responsabilidade: Remover um livro do vetor.

Entradas: livros (list), isbn (string)

Saídas: bool — True se removido, False se não encontrado

listar_livros(livros)

Responsabilidade: Exibir todos os livros cadastrados no console.

Entradas: livros (list)

Saídas: nenhuma (impressão no console)

obter_vetor_titulos(livros)

Responsabilidade: Construir um vetor unidimensional apenas com os títulos dos livros.

Entradas: livros (list)

Saídas: list — vetor de strings

4.4 Módulo validacoes.py

Módulo responsável por garantir que os dados digitados no terminal tenham o tipo e formato corretos antes de seguirem para as regras de negócio. Todas as funções repetem a pergunta ao usuário enquanto o valor informado for inválido.

ler_inteiro(mensagem, minimo, maximo)

Responsabilidade: Solicitar um número inteiro, repetindo enquanto o valor não for válido (opcionalmente dentro de um intervalo).

Entradas: mensagem (string), minimo (int, opcional), maximo (int, opcional)

Saídas: int — valor validado

ler_data_ddmmaaaa(mensagem)

Responsabilidade: Solicitar uma data DD/MM/AAAA, validar dia/mês/ano (inclusive contra o calendário real) e devolvê-la como um único inteiro DDMMAAAA.

Entradas: mensagem (string)

Saídas: int — data no formato DDMMAAAA

ler_data_ddmmaaaa_opcional(mensagem)

Responsabilidade: Mesma validação de ler_data_ddmmaaaa(), mas permite deixar em branco (usado em telas de edição).

Entradas: mensagem (string)

Saídas: int ou None

formatar_data(data_int)

Responsabilidade: Converter uma data inteira DDMMAAAA de volta para o formato DD/MM/AAAA para exibição.

Entradas: data_int (int)

Saídas: string — data formatada

ler_texto_obrigatorio(mensagem)

Responsabilidade: Solicitar um texto, repetindo enquanto o campo for deixado em branco.

Entradas: mensagem (string)

Saídas: string — texto não vazio

ler_nome(mensagem)

Responsabilidade: Solicitar um nome de pessoa, aceitando apenas letras (inclusive acentuadas) e espaços; rejeita números e símbolos.

Entradas: mensagem (string)

Saídas: string — nome válido

ler_nome_opcional(mensagem)

Responsabilidade: Mesma validação de ler_nome(), mas permite deixar em branco (usado em telas de edição).

Entradas: mensagem (string)

Saídas: string ou None

ler_status_livro(mensagem)

Responsabilidade: Solicitar o status de leitura como texto, restringindo o valor a 'Lendo', 'Lido' ou 'Quero ler' (sem diferenciar maiúsculas de minúsculas).

Entradas: mensagem (string, opcional)

Saídas: string — um de STATUS_VALIDOS

ler_status_livro_opcional(mensagem)

Responsabilidade: Mesma validação de ler_status_livro(), mas permite deixar em branco para manter o status atual (usado em telas de edição).

Entradas: mensagem (string, opcional)

Saídas: string ou None

4.5 Módulo relatorios.py

gerar_matriz_relatorio(livros)

Responsabilidade: Construir a matriz (lista de listas) do relatório de progresso.

Entradas: livros (list)

Saídas: list — matriz [titulo, status, paginas_lidas, paginas_totais, percentual]

exibir_matriz_relatorio(matriz)

Responsabilidade: Percorrer e exibir a matriz de relatório formatada no console.

Entradas: matriz (list)

Saídas: nenhuma (impressão no console)

4.6 Módulo main.py

tela_login(lista_usuarios)

Responsabilidade: Conduzir o fluxo de login/cadastro até autenticação bem-sucedida.

Entradas: lista_usuarios (list)

Saídas: dict — usuário autenticado

menu_usuarios(lista_usuarios)

Responsabilidade: Controlar o submenu de gerenciamento de usuários.

Entradas: lista_usuarios (list)

Saídas: nenhuma

menu_livros(lista_livros)

Responsabilidade: Controlar o submenu de gerenciamento de livros e relatório.

Entradas: lista_livros (list)

Saídas: nenhuma

menu_principal()

Responsabilidade: Orquestrar carregamento de dados, login e navegação entre menus.

Entradas: nenhuma

Saídas: nenhuma

5. Armazenamento dos Dados

Os dados são armazenados em dois arquivos CSV dentro da pasta `dados/`, criada automaticamente na primeira execução:

- `dados/usuarios.csv` — nome, username, senha, nascimento
- `dados/livros.csv` — titulo, autor, editora, edicao, ano, isbn, status, paginas_totais, paginas_lidas

Como o item 'organização do código' é priorizado, toda a leitura/escrita passa exclusivamente pelo módulo `arquivos.py`, o que permite trocar o formato de armazenamento (por exemplo, para um banco de dados) sem alterar as regras de negócio dos demais módulos.

6. Como Executar

Pré-requisito: Python 3 instalado. Para executar, a partir da pasta do projeto:

```
python3 main.py
```